

Project Proposal

Smart Waste Image Classification and Monitoring System

By

Rivera, Lester Van E.

2023105767

MAPUA University

In Partial Fulfillment of the Requirements for the course **CPE178P**

Foundations of AI with Huawei MindSpore and OpenHarmony OS

School of Electrical, Electronics and Computer Engineering

CPE-O | FOPI01

Mr. Dionis Padilla

Instructor

October 2026

PROJECT OVERVIEW

Background

Improper waste segregation is a persistent environmental and operational concern. Correctly identifying waste materials is an important step in supporting proper segregation and recycling practices. However, manual identification can be time-consuming and may result in inconsistent classification, particularly when different waste materials have similar visual characteristics.

The Smart Waste Image Classification and Monitoring System is proposed as a real-time computer vision application that uses a trained convolutional neural network (CNN) to classify waste items based on their visual appearance. The system will accept images captured through a USB webcam as its primary input, with image upload provided as an additional input option. The captured or uploaded image will undergo preprocessing before being passed to the trained deep learning model for classification.

The proposed system will use the TrashNet dataset, a publicly available image-classification dataset containing 2,527 images across six waste categories: glass, paper, cardboard, plastic, metal, and trash. The original dataset reports 501 glass images, 594 paper images, 403 cardboard images, 482 plastic images, 410 metal images, and 137 trash images. The images were originally collected by photographing waste objects against a white posterboard under sunlight and/or room lighting.

The dataset will be inspected and prepared specifically for this project. Data preparation will include image quality inspection, class distribution analysis, cleaning of unsuitable images, image resizing, normalization, augmentation where appropriate, and dataset splitting into training, validation, and testing subsets. The project will follow the 70% training, 15% validation, and 15% testing split specified in the CPE178P Lesson 8 Project Sprint 1 material

The initial model will be implemented using PyTorch as a baseline CNN consisting of convolutional blocks, activation functions, pooling layers, and fully connected classification layers. The baseline architecture will be intentionally kept simple so that its performance and behavior can be evaluated before additional optimization. This approach follows the Lesson 8 requirement to establish a functional baseline CNN before introducing additional complexity.

Following model development, the trained model is planned to be exported to the ONNX format and integrated with an ONNX Runtime inference workflow. A backend API will be developed using FastAPI to handle image-processing and prediction requests, while a graphical user interface (GUI) will provide users with a means of interacting with the system and viewing classification results.

The proposed project therefore establishes the foundation for the subsequent project sprints, where the initial dataset and CNN developed during Sprint 1 can be integrated with the backend, GUI, ONNX model, and eventual deployment workflow. The Lesson 8 project roadmap identifies Sprint 1 as the foundation for subsequent backend/UI integration, ONNX integration, and deployment activities.

Problem Statement

Waste materials are commonly categorized according to their physical characteristics and material type. In practical situations, identifying the appropriate waste category manually can be inefficient and may be affected by differences in human judgment, particularly when waste items have similar visual properties.

For this project, the primary engineering problem is the development of a computer vision system capable of automatically classifying a single presented waste item into one of several predefined waste categories using image data.

The proposed system addresses the following problems:

1. Manual waste identification requires human observation and can introduce inconsistencies in classification.
2. Visual similarity between waste materials can make classification difficult, particularly for objects whose appearance varies according to shape, lighting, orientation, and surface characteristics.
3. Limited automation in basic waste classification workflows creates an opportunity to apply deep learning-based image classification to assist users in identifying waste categories.
4. Model interoperability and application integration must be addressed so that the trained CNN can eventually operate as part of a complete software system rather than as an isolated training program.

Therefore, the project proposes the development of a computer vision application that combines a trained CNN model, image preprocessing, backend API services, and a graphical user interface to provide automated waste classification.

Proposed Solution

The proposed solution is a Smart Waste Image Classification and Monitoring System that receives an image of a waste item through a USB webcam or, optionally, through an image-upload function. The system will preprocess the input image and pass it to a trained CNN classification model.

The model will classify the input into one of the six waste categories represented in the selected dataset:

- Cardboard
- Glass
- Metal
- Paper
- Plastic
- Trash

The predicted category and corresponding confidence information will then be returned to the application and displayed through the graphical user interface.

The overall proposed workflow is:

Image Input → Image Preprocessing → CNN Inference → Waste Classification → Backend API → GUI Result

The system will initially focus on single-item image classification rather than multi-object detection. This scope allows the project to concentrate on CNN-based classification, model evaluation, ONNX interoperability, and real-time application integration within the available project timeframe.

Target Users

The proposed system is intended primarily for users who need assistance with identifying common waste categories through an image-based interface. Potential users include:

- Students and educational users studying artificial intelligence and computer vision;
- Individuals learning proper waste segregation;
- Demonstration users evaluating AI-based waste classification;
- Small-scale environmental or recycling initiatives requiring basic automated classification assistance; and
- Developers and researchers interested in experimenting with image-based waste classification.

The system is intended primarily as an educational and prototype computer vision application, rather than as a replacement for professional industrial waste-sorting equipment.

User Needs

The system is designed to address the following user needs:

1. Simple image input — Users should be able to provide a waste image through a USB webcam and optionally through image upload.
2. Automated classification — The system should automatically determine the predicted waste category using the trained CNN.
3. Clear classification results — The GUI should present the predicted category and relevant confidence information in an understandable format.
4. Fast inference — The system should provide prediction results within a practical response time for a prototype real-time application.
5. Accessible interaction — The interface should provide straightforward controls for capturing, submitting, or classifying an image.

6. Model-based decision support — The system should assist users in identifying the likely waste category without representing the prediction as an infallible determination.
-

Key System Requirements

The proposed system shall satisfy the following initial requirements:

Requirement ID	Requirement
SR-01	The system shall accept a waste-item image from a USB webcam.
SR-02	The system should provide an optional image-upload function.
SR-03	The system shall preprocess input images into the format required by the CNN model.
SR-04	The system shall classify an input image into one of the predefined waste categories.
SR-05	The system shall display the predicted waste category through the GUI.
SR-06	The system should display the model's prediction confidence or equivalent classification information.
SR-07	The system shall use a trained CNN model developed using PyTorch.
SR-08	The trained model shall be designed for export to the ONNX format for model interoperability.
SR-09	The system shall provide a backend API for handling prediction requests.
SR-10	The system shall provide a graphical user interface for user interaction and result presentation.
SR-11	The dataset shall undergo inspection, cleaning, preprocessing, and train/validation/test splitting before model training.
SR-12	The system shall be evaluated using appropriate classification performance metrics.

Project Scope and Limitations

The project will focus on image classification of a single primary waste item presented to the camera or provided through image upload. The initial classification categories will correspond to the six classes available in the selected TrashNet dataset which is cardboard, glass, metal, paper, plastic, and trash.

The project will not initially attempt to perform simultaneous detection and classification of multiple waste objects in a single scene. Consequently, the system will not be designed as a full object-detection or automated industrial waste-sorting system.

The initial model will be a baseline CNN Model architecture, preprocessing, augmentation, and training parameters may be refined during subsequent development based on measured performance.

The dataset's original images were collected under relatively controlled conditions, including objects placed against a white posterboard and photographed using natural or room lighting. Therefore, differences between the training dataset and real-world webcam conditions may affect classification performance. This limitation will be considered during evaluation and testing.

DETAILED USE CASE & USE CASE DIAGRAM

Use Case Overview

The Smart Waste Image Classification and Monitoring System consists of two primary operational workflows: Model Training and Waste Image Inference. Model Training represents the development workflow in which the project developer prepares the dataset, trains the convolutional neural network (CNN), evaluates its performance, and prepares the trained model for deployment. Waste Image Inference represents the end-user workflow in which a user provides a waste-item image through a USB webcam or image upload and receives a predicted waste category and corresponding confidence information.

The two primary use cases are separated because model development and model inference represent different system activities. Model Training establishes the AI model that will be used by the application, while Waste Image Inference represents the operational use of the trained model.

The main-body use-case diagrams therefore focus on these two workflows. Additional supporting use cases, detailed narratives, activity diagrams, sequence diagrams, and complete system interactions will be provided in the Appendices.

Primary Use Case 1 — Model Training

Use Case Name

Model Training

Primary Actor

Project Developer

Goal

To prepare the waste-image dataset, train a CNN classification model, evaluate its performance, and produce a trained model suitable for integration into the proposed application.

Preconditions

1. A suitable waste-image dataset is available.
2. The project development environment is configured with the required machine-learning libraries and tools.
3. The dataset contains images belonging to the defined waste categories.
4. The project developer has access to the training scripts and computational resources required for model training.

Trigger

The project developer initiates the model-development workflow after obtaining and preparing the selected waste-image dataset.

Main Success Flow

1. The project developer obtains the selected waste-image dataset.
2. The system inspects the dataset for image quality, class distribution, and data integrity.
3. The project developer performs necessary data cleaning and preparation.
4. The system preprocesses the images through operations such as resizing and normalization.
5. The dataset is divided into training, validation, and testing subsets.
6. The project developer configures the baseline CNN architecture and training parameters.
7. The system trains the CNN using the prepared training dataset.
8. The system evaluates model performance using the validation dataset during training.
9. The project developer reviews the training and validation results.
10. The trained model is evaluated using the test dataset.
11. The project developer determines whether the model meets the project's initial performance requirements.
12. The trained model is saved for subsequent inference and system integration.

Alternative / Exception Flows

- **A1 — Invalid or corrupted image:** If an image cannot be processed, the image is identified during dataset inspection and excluded or handled during data preparation.
- **A2 — Severe class imbalance:** If significant class imbalance is identified, appropriate preprocessing or training-time augmentation may be applied.
- **A3 — Poor model performance:** If the baseline model does not meet the initial performance requirements, the project developer may adjust preprocessing, training parameters, augmentation, or model architecture and repeat training.

- **A4 — Training failure:** If model training fails because of configuration, data, or computational problems, the developer corrects the identified issue and reruns the training process.

Postconditions

1. A trained CNN model is available.
 2. Training and validation results have been recorded.
 3. Test-set performance has been measured.
 4. The trained model is available for subsequent application integration and inference.
-

Primary Use Case 2 — Waste Image Inference

Use Case Name

Waste Image Inference

Primary Actor

End User

Goal

To classify a single waste item from an image and present the predicted waste category including confidence information to the user.

Preconditions

1. A trained and available classification model exists.
2. The application and required backend services are operational.
3. The user has access to a USB webcam or an image file for classification.

Trigger

The user selects the image-classification function and provides a waste-item image through the webcam or image-upload function.

Main Success Flow

1. The user opens the waste classification application.
2. The user selects the webcam or image-upload input method.
3. The user captures an image or selects an image file containing a primary waste item.
4. The system receives the image.
5. The system preprocesses the image into the input format required by the trained model.
6. The backend sends the processed input to the inference model.
7. The model performs classification.
8. The system determines the predicted waste category and corresponding confidence information.

9. The backend returns the inference result to the application interface.
10. The GUI displays the predicted waste category and confidence information to the user.

Alternative / Exception Flows

- **B1 — No image provided:** If the user does not provide a valid image, the system requests an image before performing classification.
- **B2 — Invalid image format:** If the uploaded image cannot be processed, the system informs the user and requests another image.
- **B3 — Webcam unavailable:** If the webcam cannot be accessed, the user may use the image-upload option where available.
- **B4 — Low-confidence prediction:** If the model produces a low-confidence result, the system still reports the model's prediction while indicating the associated confidence information.
- **B5 — Backend unavailable:** If the prediction service is unavailable, the system informs the user that classification cannot currently be performed.

Postconditions

1. The input image has been processed by the inference pipeline.
2. A predicted waste category is generated when inference is successful.
3. The prediction result and confidence information are displayed to the user.
4. The system remains available for another classification request.

Primary Use Case Summary

Use Case ID	Use Case	Primary Actor	Main Purpose
UC-01	Model Training	Project Developer	Prepare data, train the CNN, evaluate performance, and produce the trained model.
UC-02	Waste Image Inference	End User	Submit a waste image and receive the predicted waste category and confidence information.

Use Case Diagram — Model Training

The Model Training use case illustrates the development workflow performed by the Project Developer. The workflow includes preparing the waste-image dataset, training the baseline convolutional neural network (CNN), evaluating the model's performance, and saving the trained model for subsequent application integration and inference. These activities are represented as included use cases within the overall Model Training process.

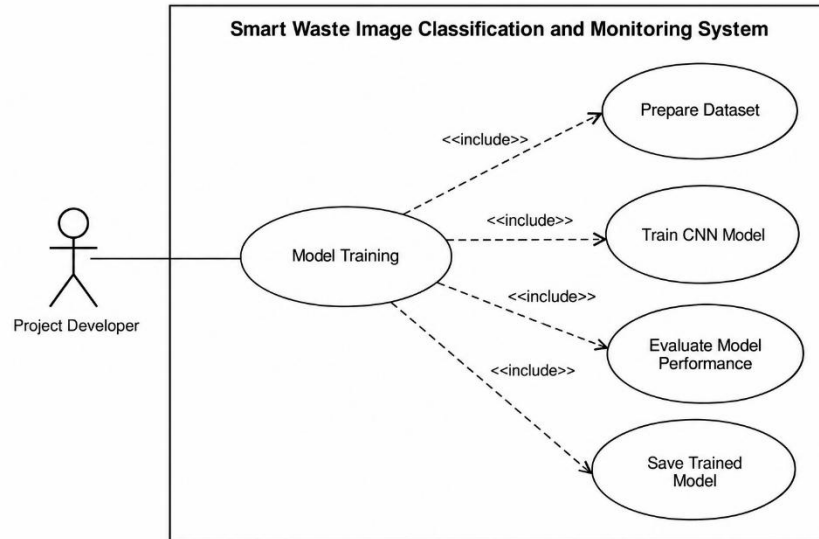


Figure 1.1 Model Training Use Case Diagram

The Use case diagram illustrating the Model Training workflow of the Smart Waste Image Classification and Monitoring System.

Figure 1 Contains:

- **Actor:** Project Developer
- **System boundary:** Smart Waste Image Classification and Monitoring System
- **Main use case:** Model Training
- **Included use cases:**
 - Prepare Dataset
 - Train CNN Model
 - Evaluate Model Performance
 - Save Trained Model
- <<include>> relationships from **Model Training** to each supporting use case.

Use Case Diagram — Waste Image Inference

The Waste Image Inference use case illustrates the operational workflow performed by the End User. The workflow includes providing a waste-item image through the USB webcam or image-upload function, preprocessing the input image, performing CNN-based classification, and displaying the predicted waste category and corresponding confidence information through the application interface.

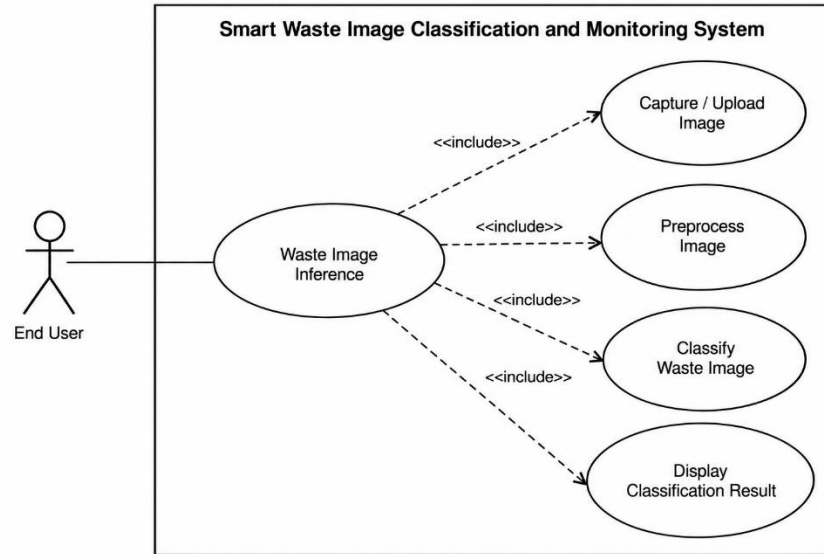


Figure 1.2 Waste Image Inference Use Case Diagram

The Waste Image Inference use case illustrates the operational workflow performed by the End User. The workflow includes providing a waste-item image through the USB webcam or image-upload function, preprocessing the input image, performing CNN-based classification, and displaying the predicted waste category including corresponding confidence information through the application interface.

Figure 2 contains:

- **Actor:** End User
 - **System boundary:** Smart Waste Image Classification and Monitoring System
 - **Main use case:** Waste Image Inference
 - **Included use cases:**
 - Capture / Upload Image
 - Preprocess Image
 - Classify Waste Image
 - Display Classification Result
-

OBJECTIVES

General Objective

The primary objective of the project is to design and develop a real-time computer vision application capable of classifying a single waste item into one of six predefined waste categories using a trained convolutional neural network (CNN), while integrating the model into a backend API and graphical user interface for practical image-based inference.

Specific Objectives

The project aims to achieve the following specific objectives:

1. To prepare and document a waste-image dataset by inspecting, cleaning, preprocessing, including organizing the selected TrashNet dataset into six classification categories which is the cardboard, glass, metal, paper, plastic, as well as trash.
 2. To implement a reproducible dataset preparation pipeline that includes image resizing, pixel normalization, appropriate data augmentation, and a 70% training, 15% validation, along with 15% testing split for CNN model development.
 3. To design and implement a baseline convolutional neural network using PyTorch with an architecture appropriate for six-class waste-image classification including verification its input dimensions, layer configuration, as well as output dimensions.
 4. To train and evaluate the baseline CNN using the prepared dataset to measure its classification performance using appropriate metrics, including accuracy, precision, recall, F1-score, as well as confusion matrix analysis.
 5. To establish an initial target of at least 80% test-set classification accuracy for the baseline model while documenting the actual performance achieved and identifying potential sources of classification error.
 6. To export the trained CNN model to the ONNX format and verify that the exported model can perform inference through an ONNX Runtime-based workflow.
 7. To develop a FastAPI backend service capable of receiving an input image, performing the required inference processing, and returning the predicted waste category to the corresponding confidence information.
 8. To develop a graphical user interface (GUI) that allows users to provide an image through a USB webcam and, where feasible, through image upload, as well as displays that resulting waste classification including confidence information.
 9. To evaluate the integrated prototype using functional test cases and model-performance measurements to verify that the major components operate correctly from image input through classification-result presentation.
 10. To document the complete development process through version-controlled source code, technical documentation, test evidence, project records, and the required AI-use disclosure.
-

METHODOLOGY

The project will follow an iterative computer vision development methodology consisting of dataset preparation, baseline CNN development, model training and evaluation, model interoperability, backend integration, graphical user interface development, and system testing. The methodology is designed to establish a functional baseline during Project Sprint 1 and progressively integrate the trained model into the complete real-time application.

The overall development pipeline is:

Dataset Collection → Dataset Inspection and Cleaning → Image Preprocessing → Dataset Splitting → CNN Architecture Design → Model Training → Model Validation and Testing → ONNX Export → Backend API Integration → GUI Integration → System Testing

Dataset Preparation

The project will use the publicly available TrashNet dataset as the initial image source for waste classification. The dataset contains six predefined waste categories which is the cardboard, glass, metal, paper, plastic, and trash.

Before model training, the dataset will undergo systematic inspection to verify image quality, class distribution, labeling consistency, and dataset integrity. Corrupted, unsuitable, or duplicate images identified during preparation will be removed or appropriately handled.

The images will then be preprocessed into a consistent format suitable for CNN training. Planned preprocessing operations include image resizing, pixel-value normalization, and appropriate data augmentation. Augmentation may include transformations such as image flipping, rotation, cropping, or other suitable variations when necessary to improve training diversity and reduce overfitting.

The prepared dataset will be divided into:

- 70% training data
 - 15% validation data
 - 15% testing data
-

Baseline CNN Development

A baseline Convolutional Neural Network (CNN) will be developed using PyTorch for the six-class waste-image classification task.

The initial architecture will consist of a sequence of convolutional layers with nonlinear activation functions and pooling operations, followed by fully connected classification layers. The final output layer will contain six output units corresponding to the six waste categories.

The baseline architecture will initially be kept relatively simple to establish a functional reference model. The model's input dimensions, intermediate tensor shapes, parameter configuration, and final output dimensions will be verified before training.

The general baseline structure will follow:

Input Image → Convolutional Block(s) → Pooling → Convolutional Block(s) → Pooling → Classification Layers → Six-Class Output

Model Training and Optimization

The prepared training dataset will be used to train the baseline CNN. The validation dataset will be used to monitor model performance during training and identify potential overfitting or underfitting.

The training process will involve configuring appropriate training parameters, including the learning rate, batch size, number of training epochs, loss function, and optimizer. These parameters will be adjusted based on observed training behavior and validation performance.

The initial training run will be the baseline of the experiment. Training and validation loss including accuracy will be recorded to evaluate whether the model is learning effectively.

If the baseline model does not achieve satisfactory performance, controlled modifications may be evaluated, such as:

- Data augmentation
- Learning-rate adjustment
- Training-parameter adjustment
- Regularization techniques
- Appropriate changes to the CNN architecture

Model modifications will be based on measured results rather than unnecessary architectural complexity.

Model Evaluation

The trained model will be evaluated using the separate test dataset that was not used during model training.

The evaluation will measure classification performance using:

- Accuracy
- Precision
- Recall
- F1-score

- Confusion matrix analysis

The confusion matrix will be used to identify which waste categories are frequently confused with one another. This analysis will help determine whether classification errors are associated with visually similar waste categories or limitations in the dataset and model. The actual experimental results will be reported after model training which were the project will not assume that the model will achieve a particular performance level before experimentation.

Model Export and Application Integration

After establishing a satisfactory trained model, the CNN will be prepared for interoperability by exporting it to the ONNX format. The exported model will subsequently be tested using ONNX Runtime to verify that the model produces valid inference results outside the original PyTorch training workflow.

The inference pipeline will accept an image from either the USB webcam or the optional image upload function. The input image will undergo the same required preprocessing used during model development before being passed to the inference model.

A FastAPI backend will be used to provide an application programming interface for receiving image inputs and returning the model's predicted waste category including confidence information.

Moreover, the graphical user interface will provide the user-facing interaction for image capture or upload and will display the resulting classification information.

System Testing

The completed prototype will undergo functional testing to verify the interaction between the image input, preprocessing pipeline, trained model, backend API, and GUI.

Testing will include successful and exceptional scenarios such as:

- Valid webcam image classification
- Valid image-upload classification
- Invalid or unsupported image input
- Unavailable webcam
- Backend or inference service failure
- Classification-result display

The system's final evaluation will consider both machine-learning performance and functional operation of the integrated application.

DATA COLLECTION AND PREPARATION

Dataset Source

The project will use the TrashNet dataset, a publicly available image classification dataset developed by Gary Thung and Mindy Yang. The dataset contains 2,527 images distributed across six waste categories which is the glass, paper, cardboard, plastic, metal, and trash. The original dataset contains 501 glass images, 594 paper images, 403 cardboard images, 482 plastic images, 410 metal images, and 137 trash images (Thung & Yang, 2016).

The images in the original dataset were collected by placing individual waste objects on a white posterboard and photographing them under sunlight and/or room lighting. The dataset's resized images have dimensions of 512×384 pixels (Thung & Yang, 2016).

The original TrashNet repository will be used as the project's primary dataset reference and will be properly cited in the project documentation.

Dataset Categories

The dataset will be used as a six-class classification problem. The target categories are:

Class	Number of Images
Cardboard	403
Glass	501
Metal	410
Paper	594
Plastic	482
Trash	137
Total	2,527

The distribution shows that the dataset is not perfectly balanced, with the trash category having substantially fewer images than the other categories. This imbalance will be considered during dataset analysis and model training.

Dataset Inspection and Cleaning

Before model training, the dataset will be inspected to verify its suitability for the proposed classification task. The inspection will examine image availability, image format, image dimensions, class distribution, and general image quality.

The following preparation activities will be performed:

1. Verify that images can be opened and processed correctly.
2. Identify corrupted or unreadable image files.
3. Check that images are stored under the correct class labels.
4. Examine the number of images available for each class.
5. Identify unsuitable images that may interfere with model training.
6. Check for duplicate or highly redundant images where feasible.
7. Record the results of the dataset inspection for documentation.

Images identified as corrupted or unsuitable will be removed or excluded from the training pipeline rather than being used as training samples.

Image Preprocessing

The cleaned images will undergo preprocessing before being provided to the CNN. Preprocessing will ensure that input images have a consistent representation and are compatible with the model architecture.

The planned preprocessing pipeline consists of:

Input Image → Resize → Convert to Tensor → Normalize → CNN Input

The images will be resized to a consistent input resolution selected during model implementation. Pixel values will then be normalized using a consistent transformation applied during both training and inference.

The training time data augmentation may also be applied to increase the diversity of training samples. Appropriate transformations may include limited rotation, horizontal flipping, cropping, and other controlled image transformations that do not change the underlying waste category.

The same fundamental preprocessing requirements used during model training will be maintained during application inference to ensure consistency between the training and deployment pipelines.

Dataset Splitting

After cleaning and preprocessing, the dataset will be divided into three subsets:

- 70% Training Set — used to train the CNN model.
- 15% Validation Set — used to monitor model performance during training and support model and hyperparameter selection.
- 15% Testing Set — reserved for final performance evaluation after model training.

The project will use a reproducible splitting procedure so that the dataset organization can be recreated during development. Where appropriate, the splitting process will preserve the relative representation of the six classes to reduce the effect of the existing class imbalance.

MODEL DEVELOPMENT

Model Development Approach

The project will develop a baseline convolutional neural network (CNN) for six-class waste image classification. The model will be implemented using PyTorch and trained using the prepared TrashNet dataset. The initial model will prioritize a simple and reproducible architecture that can be trained and evaluated efficiently before additional optimization is considered.

The development process will follow the sequence:

Prepared Dataset → CNN Architecture → Model Training → Validation → Model Optimization → Final Testing → Model Export

The baseline model will first be established before introducing more complex modifications. This approach allows the performance of the initial model to serve as a reference for evaluating subsequent improvements.

CNN Architecture

The proposed baseline CNN will consist of three convolutional blocks, followed by fully connected layers for classification. Each convolutional block will extract increasingly complex visual features from the input image.

The planned architecture is:

1. Input Layer – receives a preprocessed RGB image resized to a consistent input resolution.
2. Convolutional Block 1 – convolution, ReLU activation, and max-pooling for initial feature extraction.
3. Convolutional Block 2 – convolution, ReLU activation, and max-pooling for intermediate feature extraction.
4. Convolutional Block 3 – convolution, ReLU activation, and max-pooling for higher-level feature extraction.
5. Flattening Layer – converts the extracted feature maps into a one-dimensional feature vector.
6. Fully Connected Layer – learns relationships among the extracted features.
7. Output Layer – produces six class scores corresponding to cardboard, glass, metal, paper, plastic, and trash.

The baseline architecture is intentionally kept relatively simple so that its training behavior and classification performance can be clearly evaluated before further optimization. The exact number

of filters and other architectural parameters may be adjusted during implementation based on computational requirements including validation performance.

Training Configuration

The CNN will be trained using the training portion of the prepared dataset, while the validation set will be used to monitor model performance during training.

The initial training configuration will use:

Parameter	Planned Configuration
Framework	PyTorch
Task	Six-class image classification
Model	Baseline CNN
Loss Function	Cross-Entropy Loss
Optimizer	Adam
Initial Learning Rate	0.001
Batch Size	32
Epochs	Approximately 20–30
Validation	15% validation set
Model Selection	Best validation performance

These values represent the initial training configuration, rather than fixed final values. Learning rate, batch size, number of epochs, and other hyperparameters may be adjusted after the initial training run based on validation performance as well as computational constraints.

Because the dataset contains unequal numbers of images among the six classes, class imbalance will also be considered during training. If the initial results indicate that minority classes, particularly the trash category, are being poorly recognized, techniques such as class-weighted loss or weighted sampling may be evaluated.

Model Training and Optimization

During training, the CNN will learn the visual features associated with each waste category by minimizing the classification loss between the predicted class and the ground-truth label.

Training and validation performance will be recorded for each epoch. The following information will be monitored:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy

Training and validation curves will be generated to help identify possible underfitting or overfitting. If the baseline model does not meet the desired performance level, controlled modifications may be evaluated, such as data augmentation, regularization, learning-rate adjustment, class balancing, or changes to the CNN architecture. On the other hand, the optimization process will focus on improving generalization to previously unseen waste images rather than simply increasing training-set accuracy.

Model Evaluation and Selection

After training, the model will be evaluated using the separate test dataset. The test set will not be used to directly optimize the model during training.

The following metrics will be used:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

Performance will also be examined on a per-class basis to determine whether the model performs consistently across the six waste categories. Particular attention will be given to classes with fewer training samples.

The best-performing model checkpoint based on validation performance will be selected as the candidate model for final testing and application integration. Actual performance values will only be reported after the model has been trained and tested during implementation.

Model Saving and Export

The selected PyTorch model will be saved as a trained model checkpoint for reproducibility and later integration. The model will subsequently be exported to the ONNX format to provide an interoperable representation for application deployment.

The exported model will be tested using ONNX Runtime to verify that it can perform inference correctly outside the original PyTorch training environment. The outputs of the ONNX model will be compared with the corresponding PyTorch model outputs using the same test inputs.

The model development pipeline can therefore be summarized as:

TrashNet Dataset → Preprocessing → Baseline CNN → Training → Validation → Optimization → Testing → PyTorch Model → ONNX Export → ONNX Runtime Verification

This model will classify the component of the proposed Smart Waste Image Classification and Monitoring System then will subsequently be integrated with the backend API along with graphical user interface that described in later sections.

SOFTWARE INTERFACE

System Interface Overview

The proposed system will provide a graphical interface that allows an end user to submit a waste image and receive the classification result generated by the trained CNN model. The interface will be the primary interaction point between the user and the image classification system.

The software interface will follow the processing flow:

Image Input → Image Preprocessing → Backend API → CNN/ONNX Runtime Inference → Classification Result → User Interface

The interface will support two image-input methods:

- USB Webcam Capture – captures an image of a waste item using the connected webcam.
- Image Upload – allows the user to select an existing image file for classification.

The webcam will be the primary input method, while image upload will provide an alternative for testing and demonstration.

User Interface Components

The graphical interface will contain the following essential components:

Component	Function
Camera/Input Area	Displays the webcam feed or selected image
Capture Button	Captures an image from the USB webcam
Upload Button	Allows the user to select an image file
Classify Button	Sends the selected image for classification
Result Display	Shows the predicted waste category

Component	Function
Confidence Information	Displays the model's prediction confidence
Status/Message Area	Provides feedback about processing or input errors

The interface will be designed to keep the classification workflow straightforward. The user should be able to provide an image, initiate classification, and view the result without requiring knowledge of the underlying machine-learning model.

Backend API

A FastAPI-based backend will provide the communication layer between the graphical interface and the trained classification model.

The backend will receive an image from the interface, perform or invoke the required preprocessing, execute model inference, and return the classification result to the interface.

The planned workflow is:

GUI → FastAPI Endpoint → Image Preprocessing → ONNX Runtime → Prediction → FastAPI Response → GUI

The API will be designed around the primary classification operation. A classification request will contain an image, while the response will provide the predicted waste category and associated confidence information.

This separation between the interface, API, and model inference component will allow the machine-learning model to operate independently from the user interface.

Model Integration

The trained model developed in Section 7 will be exported to ONNX and integrated into the backend using ONNX Runtime. The same preprocessing assumptions used during model development will be maintained during application inference to ensure that images supplied through the interface are processed consistently.

The inference component will:

1. Receive the input image.
2. Validate the image input.
3. Resize and preprocess the image according to the trained model requirements.
4. Convert the image into the required model input format.
5. Execute inference using ONNX Runtime.

6. Identify the class with the highest predicted score.
7. Obtain the corresponding confidence information.
8. Return the result to the graphical interface.

The six possible classification outputs are:

- Cardboard
 - Glass
 - Metal
 - Paper
 - Plastic
 - Trash
-

Error Handling

The interface and backend will provide basic error handling for common input and system conditions. These may include:

- No image provided.
- Unsupported or invalid image file.
- Webcam unavailable or disconnected.
- Failure to capture an image.
- Invalid image format.
- Backend/API connection failure.
- Model inference failure.

When an error occurs, the system should provide a clear status message rather than displaying an incomplete or misleading classification result.

Interface and System Integration

The software interface will be integrated with the model through the FastAPI backend. This architecture separates the major system components into distinct layers:

- **Presentation Layer:** Graphical user interface for image input and result display.
- **Application/API Layer:** FastAPI backend responsible for receiving requests and coordinating image preprocessing and inference.
- **AI/Inference Layer:** ONNX model executed through ONNX Runtime to classify the submitted waste image.

This separation supports modular development and makes it possible to modify the interface without changing the underlying classification model. It also allows the model and backend components to be tested independently before complete system integration.

The final interface will be evaluated based on its ability to successfully capture or receive an image, process the request, obtain a classification result, and clearly present the predicted waste category including confidence information to the user.

MODEL PERFORMANCE EVALUATION

Evaluation Approach

The performance of the Smart Waste Image Classification and Monitoring System will be evaluated at both the machine-learning model level and the integrated system level. Model evaluation will determine how accurately the CNN classifies unseen waste images, while system evaluation will determine whether the complete application can successfully receive an image, process it, perform inference, and present the result to the user.

The evaluation process will use the separate test dataset that was established during dataset preparation. The test dataset will not be used during model training or hyperparameter optimization to maintain an independent evaluation of the final model.

The overall evaluation process is:

Trained Model → Test Dataset → Model Predictions → Performance Metrics → Error Analysis
→ System Integration Testing

Classification Metrics

The trained model will be evaluated using several classification metrics to provide a more complete assessment than accuracy alone.

Accuracy

Accuracy measures the proportion of correctly classified images among all test images.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

For the six-class classification problem, overall accuracy will provide a general measure of the model's classification performance.

Precision

Precision measures how many of the images predicted as a particular class actually belong to that class.

$$Precision = \frac{TP}{TP + FP}$$

Precision will help determine whether the model frequently assigns images to a class incorrectly.

Recall

Recall measures how many of the actual images belonging to a particular class are correctly identified

$$Recall = \frac{TP}{TP + FN}$$

Recall is particularly important for identifying whether the model is failing to recognize examples belonging to a particular waste category.

F1-Score

The F1-score combines precision and recall into a single metric.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

The F1-score will be examined on a per-class basis and, where appropriate, using an aggregate averaging method to account for the six classification categories.

Confusion Matrix

A confusion matrix will be generated to visualize the classification results across the six waste categories:

- Cardboard
- Glass
- Metal
- Paper
- Plastic
- Trash

The matrix will show which classes are correctly classified and which classes are commonly confused with one another. This will allow the project developer to identify specific classification problems that may not be apparent from overall accuracy alone.

For example, visually similar materials may be incorrectly classified as one another. Such errors will be examined to determine whether additional preprocessing, augmentation, class balancing, or model adjustments should be considered.

Target Performance

The project will use 80% test-set accuracy as an initial engineering target for the baseline system.

This value represents a development target rather than a guaranteed result. The actual accuracy, precision, recall, F1-score, and confusion matrix will only be reported after the model has been trained as well as evaluated using the final test dataset.

If the initial model does not meet the target, the project will investigate possible causes and evaluate controlled improvements such as:

- Data augmentation
- Class balancing
- Hyperparameter adjustment
- Regularization
- CNN architecture modification
- Additional training

The final reported results will reflect the actual experimental performance rather than predetermined values.

System Functional Evaluation

In addition to model performance, the complete application will be tested to verify that the individual software components operate correctly together.

The following functional scenarios will be evaluated:

Test Area	Expected Result
Webcam capture	The system successfully captures an image from the connected USB webcam.
Image upload	A valid supported image can be selected and submitted.
Image preprocessing	The input is converted into the format required by the model.
Classification request	The backend successfully receives and processes the image.

Test Area	Expected Result
Model inference	ONNX Runtime returns a classification result.
Result display	The predicted waste category and confidence information are displayed.
Invalid input	The system provides an appropriate error message.
Webcam unavailable	The system indicates that the webcam cannot be accessed.
Backend failure	The interface provides appropriate feedback when the API cannot process the request.

Error Analysis

Incorrect predictions identified during testing will be analyzed to determine common sources of classification error. The analysis may consider factors such as:

- Similar visual characteristics between waste categories.
- Insufficient representation of a class.
- Image quality and lighting conditions.
- Object orientation or position.
- Background differences between the training dataset and webcam images.
- Classification ambiguity between visually similar materials.

The results of this analysis will guide any necessary improvements to the model or preprocessing pipeline.

Evaluation Results and Documentation

The final evaluation will document the model's test accuracy, precision, recall, F1-score, and confusion matrix, together with the results of the system functional tests.

Training and validation curves will also be retained as supporting evidence of model behavior during development. Any modifications made after the baseline model will be documented so that their effects on performance can be compared.

The evaluation results will provide the basis for determining whether the trained model and integrated application meet the project's intended functional including performance objectives.

EXPECTED OUTCOMES

Functional Prototype

The project is expected to produce a functional prototype of the Smart Waste Image Classification and Monitoring System capable of accepting an image of a single waste item through a USB webcam or image upload. The system should process the input image and provide a corresponding waste classification through the graphical interface.

The prototype is expected to demonstrate the complete workflow:

Image Input → Preprocessing → CNN/ONNX Inference → Waste Classification → Result Display

Trained Waste Classification Model

A trained CNN model is expected to classify waste images into the six predefined TrashNet categories:

- Cardboard
- Glass
- Metal
- Paper
- Plastic
- Trash

The project is expected to produce a reproducible trained model together with its training configuration, evaluation results, and supporting performance visualizations.

The initial engineering target is to achieve at least 80% test-set accuracy. However, the actual performance will be determined through the final evaluation and will not be assumed in advance.

Model Evaluation Results

The project is expected to produce quantitative evidence of model performance through:

- Test accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Training and validation curves

These results will be used to determine the strengths and weaknesses of the classification model and identify categories that may require further improvement.

Integrated Application

The trained model is expected to be integrated into a software application consisting of a graphical interface and backend API.

The integrated system is expected to:

1. Receive an image from the USB webcam or image upload.
2. Validate and preprocess the image.
3. Send the image to the backend API.
4. Execute inference using the exported ONNX model through ONNX Runtime.
5. Determine the predicted waste category.
6. Provide confidence information.
7. Display the classification result to the user.

The integration will demonstrate that the trained model can operate as part of an application rather than only as an isolated machine-learning experiment.

ONNX Model and Interoperability

The project is expected to produce an ONNX version of the trained model and verify its operation using ONNX Runtime. The exported model should produce consistent classification behavior when compared with the corresponding PyTorch model using the same input.

This will provide a deployment-ready model representation that is separated from the original training environment.

Documentation and Reproducibility

The project is expected to produce sufficient technical documentation to allow the development process to be understood and reproduced. This will include:

- Dataset source and preparation procedure
- Dataset split and preprocessing configuration
- CNN architecture
- Training configuration
- Model evaluation results
- Software architecture
- API and interface integration
- Testing results
- GitHub repository and commit history
- AI-use disclosure and development documentation

The resulting repository will contain the relevant source code, configuration files, documentation, and other project artifacts required to demonstrate the development process.

Expected Overall Outcome

The final expected outcome is a working proof of concept computer vision system that demonstrates the practical application of a CNN-based image classification model to waste identification. The project will combine dataset preparation, model development, performance evaluation, model interoperability, backend integration, and graphical user interaction into a single system.

The project is intended as an educational and prototype level solution which is not expected to replace industrial waste sorting systems or provide multi-object waste detection. Its primary outcome is to demonstrate a complete and reproducible AI application capable of classifying a single waste item into one of six predefined categories.

Project Timeline

CPE178P – Smart Waste Image Classification and Monitoring System

Project Schedule / Gantt Chart – Lester Van E. Rivera (Solo Project)

Schedule aligned with the currently visible Blackboard deadlines: Project Proposal – Oct. 4, Sprint 1 – Oct. 7, Sprint 2 – Oct. 11, Final Project – Oct. 11, 2026.

WBS	Task Title	Task Owner	Estimate (hrs)	Complete (hrs)	Remaining (hrs)	Sprint	Start Date	Due Date	Duration (days)	% Complete	Status	4	5	6	7	8	9	10	11
1.0	Project Proposal and Project Setup	Rivera, L.	12	6	6	6 Sprint 1	10/04/26	10/07/26	4	50% In Progress		■	■	■	■				
1.1	Finalize Project Proposal	Rivera, L.	6	6	0	0 Sprint 1	10/04/26	10/07/26	4	100% Complete		■	■	■	■				
1.2	Set Up Development Environment and Dependencies	Rivera, L.	2	0	2	2 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
1.3	Create GitHub Repository and Initial Commit	Rivera, L.	2	0	2	2 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
1.4	Prepare Sprint 1 Evidence and Documentation	Rivera, L.	2	0	2	2 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
2.0	Dataset Preparation and Baseline Deep Learning Model	Rivera, L.	24	0	24	24 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
2.1	Acquire and Inspect TrashNet Dataset	Rivera, L.	4	0	4	4 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
2.2	Clean, Preprocess, and Split Dataset	Rivera, L.	5	0	5	5 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
2.3	Implement Baseline CNN	Rivera, L.	6	0	6	6 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
2.4	Train and Evaluate Baseline Model	Rivera, L.	6	0	6	6 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
2.5	Record Sprint 1 Development Evidence	Rivera, L.	3	0	3	3 Sprint 1	10/04/26	10/07/26	4	0% Not Started		■	■	■	■				
3.0	Project Sprint 2 and Final System Integration	Rivera, L.	38	0	38	38 Sprint 2	10/08/26	10/11/26	4	0% Not Started						■	■	■	■
3.1	Refine/Finalize Deep Learning Model	Rivera, L.	6	0	6	6 Sprint 2	10/08/26	10/11/26	4	0% Not Started						■	■	■	■
3.2	Integrate FastAPI Backend	Rivera, L.	7	0	7	7 Sprint 2	10/08/26	10/11/26	4	0% Not Started						■	■	■	■
3.3	Integrate Flet Graphical Interface	Rivera, L.	7	0	7	7 Sprint 2	10/08/26	10/11/26	4	0% Not Started						■	■	■	■
3.4	Integrate Image Input and Classification Workflow	Rivera, L.	5	0	5	5 Sprint 2	10/08/26	10/11/26	4	0% Not Started						■	■	■	■
3.5	ONNX Export/Runtime Verification	Rivera, L.	4	0	4	4 Sprint 2	10/08/26	10/11/26	4	0% Not Started						■	■	■	■
3.6	System Testing and Debugging	Rivera, L.	5	0	5	5 Sprint 2	10/08/26	10/11/26	4	0% Not Started						■	■	■	■
3.7	Final Documentation, README, and Demonstration Preparation	Rivera, L.	4	0	4	4 Sprint 2	10/08/26	10/11/26	4	0% Not Started						■	■	■	■

Key Submission Milestones

Project Proposal 10/04/2026
Project Sprint 1 – Individual Submission 10/07/2026

Project Sprint 1 – Group Deliverables 10/07/2026
Project Sprint 2 – Individual Submission 10/11/2026
Project Sprint 2 – Group Deliverables 10/11/2026

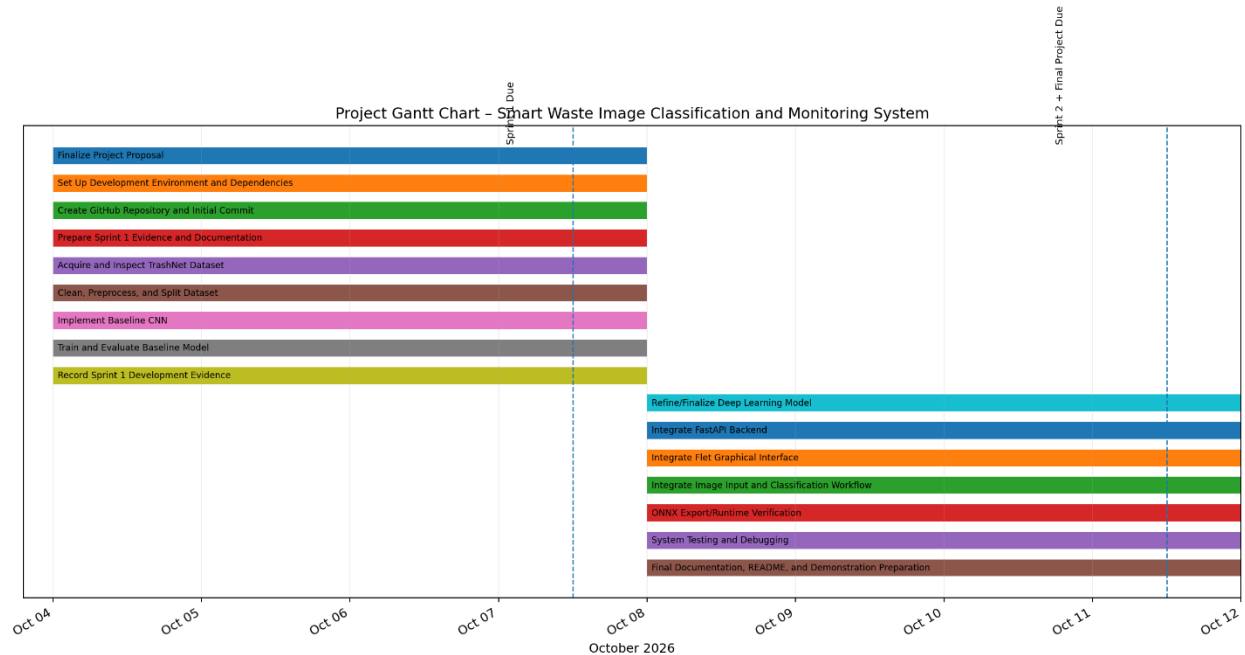
Final Project Deliverables 10/11/2026

Proposal PDF / required schedule
Individual Sprint 1 evidence and contribution

Group deliverables; solo project still follows required artifacts

Sprint 2 individual submission

Sprint 2 group deliverables
Presentation video, documentation, source code + README



GITHUB PROJECT REPOSITORY

Repository Purpose

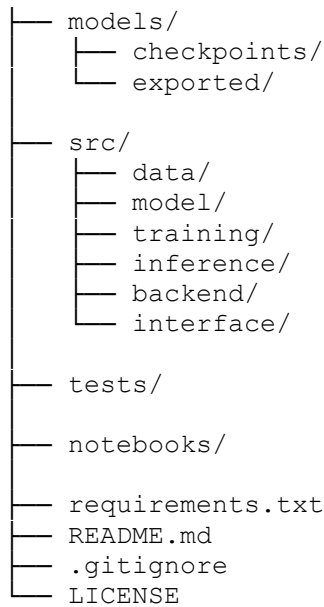
A GitHub repository will be used as the central version control as well as source code repository for the Smart Waste Image Classification and Monitoring System. The repository will contain the project's source code, dataset preparation scripts, model development files, backend and interface components, configuration files, documentation, including other relevant project artifacts.

The repository will also provide a record of the project's development progress through Git commits. This will support project documentation, contribution tracking, reproducibility, and verification of the development process.

Proposed Repository Structure

The repository will be organized into separate directories according to the major components of the project.

```
smart-waste-image-classification/  
├── data/  
│   ├── raw/  
│   ├── processed/  
│   └── README.md
```



The actual directory structure may be adjusted during implementation depending on the final software architecture and framework requirements.

Version Control and Development Tracking

Git will be used to track changes throughout the development process. Commits will be organized according to meaningful development activities, such as dataset preparation, model implementation, training, backend development, interface development, integration, testing, and documentation.

Examples of commit activities include:

- Initial project structure and environment setup
- Dataset preparation and preprocessing
- Baseline model implementation
- Model training and evaluation
- Backend API implementation
- Flet interface implementation
- Model export and integration
- System testing and debugging
- Documentation updates

As the project is being developed individually, the GitHub commit history will also be evidence of the for individual contribution and development progress.

Repository Documentation

The repository will contain a README.md file describing the project including providing instructions for setting up and running the system. The README will include, as applicable:

- Project overview
- System requirements
- Installation instructions
- Required Python packages and dependencies
- Dataset preparation instructions
- Model training instructions
- Model inference instructions
- Backend/API setup
- Flet interface setup
- Project directory structure
- Testing instructions

Sensitive information, unnecessary generated files, and large dataset files that should not be stored directly in the repository will be excluded using .gitignore along with appropriate repository practices.

GitHub Repository Deliverable

The final repository will be the project's source code reference and development record. The repository link will be included in the final project documentation and submission materials. Moreover, the repository will be maintained throughout the project so that the development history remains traceable from the initial setup through model development, system integration, testing, and final implementation.

APPENDICES

Appendix A. Detailed Use Case Diagrams and Narratives

Appendix B. Detailed System and Class Diagrams

Appendix C. Activity Diagrams

Appendix D. Sequence Diagrams

Appendix E. Requirements Traceability Matrix

Appendix F. System Test Cases

Appendix G. AI Use and Disclosure Log

Appendix H. Supporting Project Materials

REFERENCES

- Thung, G., & Yang, M. (2016). Classification of trash for recyclability status. Stanford University, CS229 Project. <https://github.com/garythung/trashnet>
- ONNX. (2026). ONNX documentation. <https://onnx.ai/onnx/>
- ONNX Runtime. (2026). ONNX Runtime documentation. <https://onnxruntime.ai/docs/>
- FastAPI. (2026). FastAPI documentation. <https://fastapi.tiangolo.com/>
- Flet. (2026). Flet documentation. <https://flet.dev/docs/>